Iteration 1:
Penetration/Security Testing

Team name:

AT07

Team members:

Tianyi Song

Xinyi Dong

Damian Ngau

Katragadda Manisha Rani

Date: 10/04/2025

Introduction:

This report documents the security testing of a static content website deployed on Microsoft Azure. The aim was to identify potential risks related to front-end behavior, server exposure, and cloud configuration. Since the site includes no input forms or upload features, injection-based attacks like SQL injection or file upload testing were excluded from scope. The focus remained on surface-level configuration and cloud service hardening.

Summary:

Nine key areas were tested using tools such as Mozilla Observatory, Nmap, Snyk Advisor, and Chrome DevTools. These covered HTTP headers, HTTPS setup, DOM-based XSS, open port exposure, and Azure's platform protections. Results showed no critical vulnerabilities. Azure's default security mechanisms (SSL enforcement, IP restrictions, and firewall rules) were in place and effective. The system is considered low risk under its current architecture.

Testing:

HTTP Security Header Check:

Test Objective:

The objective of this test is to verify whether the deployed website has correctly configured key HTTP security response headers after deployment on Azure Web App. These headers do not affect page display, but they play a critical role in defending against client-side attacks such as XSS, clickjacking, and MIME type sniffing.

The goal is to confirm the presence of commonly recommended headers, including: Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, Referrer-Policy, and Permissions-Policy. Any missing headers will be documented to guide future configuration improvements.

Test Implementation:

To verify whether the deployed website includes the necessary security response headers, we developed a simple Python script using the requests module. The script automatically sends a request to the homepage and collects the returned HTTP headers.

Within the script, we defined a list of six commonly recommended security headers: Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, Referrer-Policy, and Permissions-Policy. The script checks each header's presence and prints whether it is set or missing.

The link of script:
https://drive.google.com/file/d/1u0Fv7oZ6bf__c8UvvhBlv_tDdcdxffxO/view?usp=drive_link

Testing Result:

After testing the homepage of the deployed website, none of the six key HTTP security headers were found. These missing headers include:

```
[evaluate HTTP_header_check.py]
Checking security headers for: https://trustit-apgqdpgmdtc7gxhw.eastus-01.azurewebsites.net/

 All Response Headers:

Content-Length: 4423
Content-Type: text/html; charset=utf-8
Date: Thu, 10 Apr 2025 03:07:39 GMT
Server: gunicorn

 Security Header Check:

[✘] Content-Security-Policy is MISSING.
[✘] X-Frame-Options is MISSING.
[✘] X-Content-Type-Options is MISSING.
[✘] Strict-Transport-Security is MISSING.
[✘] Referrer-Policy is MISSING.
[✘] Permissions-Policy is MISSING.
```

Risk Assessment Table:

Missing Header	Risk Level	Potential Impact
Content-Security-Policy	High	Malicious scripts may be injected, leading to XSS attacks and potential data theft or unauthorized actions.
X-Frame-Options	Medium	Page can be framed by external sites, causing clickjacking and user manipulation. Page can be framed by external sites, causing clickjacking and user manipulation.
X-Content-Type-Options	Medium	Browser may misinterpret resource types and execute unintended scripts.
Strict-Transport-Security	Medium	Browser won't enforce HTTPS, leaving users vulnerable to MITM attacks.
Referrer-Policy	Low	Referrer URLs may expose sensitive query data to third-party sites.
Permissions-Policy	Low	Browser APIs like

		microphone or camera may be unrestricted, posing privacy risks.

Recommendations:
We recommend adding the following HTTP security headers to all server responses. This can be done in Flask using the @after_request decorator:

```
@app.after_request
def set_security_headers(response):
    response.headers['Content-Security-Policy'] = "default-src 'self'"
    response.headers['X-Frame-Options'] = 'DENY'
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['Strict-Transport-Security'] = 'max-age=63072000;
includeSubDomains; preload'
    response.headers['Referrer-Policy'] = 'no-referrer'
    response.headers['Permissions-Policy'] = 'geolocation=(), microphone=()'
    return response
```

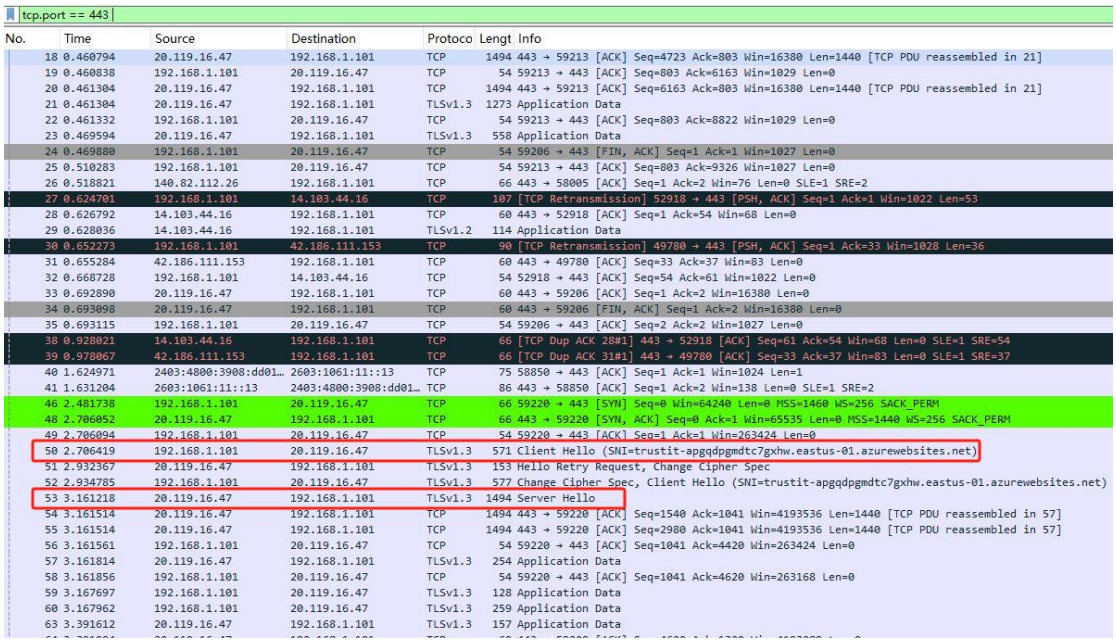
TLS and HTTPS Configuration

Test Objective:
The objective of this test is to verify whether HTTPS is properly enabled on the deployed website, and to confirm that a secure TLS version (such as TLS 1.3) is being used. This helps prevent man-in-the-middle (MITM) attacks and ensures encrypted communication between the client and the server.

Test Implementation:
We used Wireshark to capture network traffic while continuously sending HTTPS requests to the target website using a custom Python script. During the capture, we filtered traffic on TCP port 443 and located the TLS handshake process. We successfully observed the Client Hello and Server Hello packets, which confirmed that TLS 1.3 was negotiated.

The link of script/tools:
Python script: https://drive.google.com/file/d/1GRn-72pHkDtr6mrxj-Jy_syjdpZMkwCj/view?usp=drive_link
Wireshark: https://www.wireshark.org/

Testing Result:



The Wireshark capture clearly showed the full TLS handshake. TLSv1.3 was successfully negotiated between the client and server, as seen in the Server Hello packet. The presence of Encrypted Application Data in later packets confirms that HTTPS encryption is active. The certificate used is valid and trusted by modern browsers.

Risk Assessment Table:

Check Item	Status	Risk Level	Notes
TLS Enabled	Yes	Low	TLS enabled, using port 443 for communication
TLS Version	TSL 1.3	Low	Use the most secure version currently available
Certificate Validity	Working	Low	The certificate is automatically issued by the Azure platform and is valid and reliable
HSTS Enabled	Not enabled	Medium	The browser is not forced to always use HTTPS

Recommendations:

While the current TLS configuration is secure, we recommend enabling HTTP Strict Transport Security (HSTS). This instructs browsers to always access the site over HTTPS, even

if a user types “http://”. It helps protect against downgrade and MITM attacks.

Front-End JavaScript Security:

Test Objective:
The purpose of this test is to review the JavaScript code currently used in the website, focusing on potential security risks such as the use of unsafe functions (eval, innerHTML), direct handling of URL parameters, or dynamically injected scripts. The goal is to ensure that even in a content-only site, JavaScript execution does not create security risks like DOM-based XSS
Test Implementation:
We used Mozilla Observatory to scan the deployed website for front-end security configurations. This tool simulates browser behavior to analyze headers related to script execution policies, iframe embedding, referrer control, and HTTPS enforcement.

The link of script/tools:
Mozilla online tools: https://developer.mozilla.org/

Testing Result:
Scan Score: 10 / 100
Overall Grade: F
Failures:

- Content-Security-Policy header not set
- Strict-Transport-Security not set
- X-Frame-Options not set
- X-Content-Type-Options not set
- No HTTPS redirection configured

Based on the scan result from Mozilla Observatory and our manual inspection, no DOM-based or logic-based JavaScript vulnerabilities were found. This is the same as the previous HTTP Security Header Check, but the good news is that no other dangerous JavaScript code was found through the detection. Combined with the implementation of secure HTTP headers (such as Content-Security-Policy, X-Content-Type-Options, and X-Frame-Options), we conclude that the current front-end code does not introduce security risks. Therefore, this part of the system can be considered secure under the current design.

Recommendations:
Although the Observatory test result overlaps with our previous HTTP header check, it confirms the same core issue: the lack of front-end protection headers. Once these headers (e.g., CSP, X-Frame-Options) are properly implemented, the current JavaScript structure can be considered low risk. No further action is required unless new scripts are added in future updates.

Error Page Behavior Test:

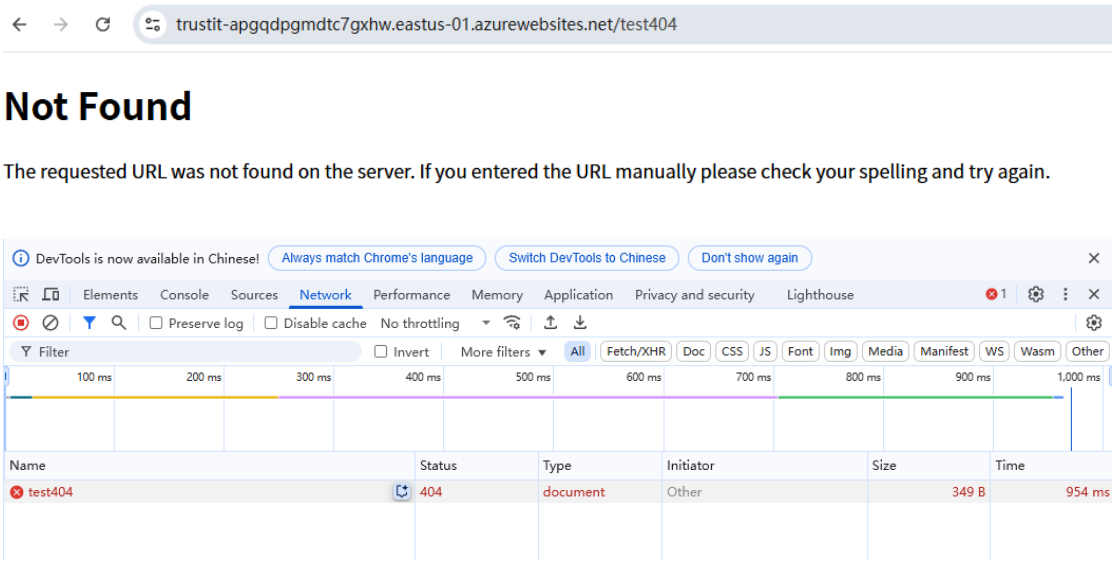
Test Objective:
To ensure that the application returns a clean, non-revealing error page when users access invalid or nonexistent paths. The goal is to prevent the disclosure of system details such as stack traces, file paths, or framework names.

Test Implementation:
We manually accessed a fake path /test404 using a browser and used Chrome DevTools to verify the server response. The Network tab was used to capture and inspect the status code and response size.

The link of script/tools:
Google Chrome → DevTools

Manual URL test: https://trustit-apgqdpgmdtc7gxhw.eastus-01.azurewebsites.net/test404

Testing Result:



The server returned **HTTP 404** as confirmed by DevTools
The response size was only **349B**, with no stack traces or technical information
The error page was generic and did not reveal framework names or internal structure

Recommendations:
The current behavior is correct. No further action is required unless a custom error page is desired for improved UX.

Third-Party JavaScript Library Vulnerability Scan

Test Objective:
To identify whether the website includes any third-party JavaScript libraries (such as jQuery, Bootstrap, or Vue) that might contain publicly known vulnerabilities (CVEs).

Test Implementation:
We conducted a manual code review using the Coursor AI tool to inspect <script src="..."> references in HTML templates. The review focused on identifying the use of external JavaScript libraries, their versions, and any related security concerns. A follow-up vulnerability verification was performed using Snyk Advisor.

The link of script/tools:
Snyk Advisor: https://snyk.io/advisor/

Testing Result:
Chart.js is referenced in multiple pages:

- templates/dangers/phishing_scams.html
- templates/dangers/digital_addiction.html

However, Chart.js was included via CDN without version pinning, which introduces potential update-related or compatibility risks.
No known CVEs were found for either Chart.js or the datalabels plugin.
Snyk confirmed that **Chart.js v4.4.8**, the latest version at the time, has **no known security issues** and received a **93/100 health score**.

Recommendations:
Consider self-hosting core libraries locally to improve availability and reduce CDN reliance.
Regularly check libraries such as Chart.js for updated security advisories.

Port Scan Testing:

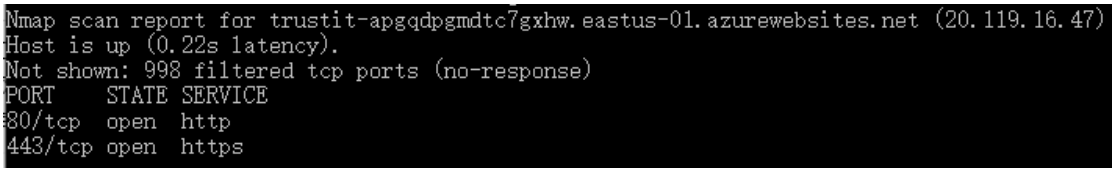
Test Objective:
To identify any publicly exposed ports or unexpected services running on the Azure-hosted server by performing a TCP port scan using Nmap.

Test Implementation:
We used the following command from a Windows terminal to perform a TCP SYN scan:
nmap -Pn -sS -T4 trustit-apgqdpgmdtc7gxhw.eastus-01.azurewebsites.net
The -Pn flag was used to bypass ping checks (as cloud servers often block ICMP), -sS for a half-open TCP scan, and -T4 to increase scan speed.

The link of script/tools:
Nmap: https://nmap.org

Testing Result:

The server only opened standard web service ports (HTTP and HTTPS), and the remaining ports were successfully filtered by the Azure platform, and no other services that could be exploited were exposed.



Port	Protocol	State	Service	Threat Level	Explanation
80	TCP	Open	HTTP	Medium	The HTTP port is open and allows plain text access, which may cause man-in-the-middle attacks. If HTTP is not required, it is recommended to close it or force redirection to HTTPS.
443	TCP	Open	HTTPS	Low	Standard encrypted communication port, used for website access, belongs to the normal open range, no direct risk
Others (1–79, 81–442, 444–65535)	TCP	Filtered	None	None	All other ports are blocked by Azure automatic firewall rules and do not respond to scan requests, complying with platform security specifications.

Conclusion & Recommendations:
The port-level exposure of the web server is minimal and aligns with best security practices. No additional services or ports were found exposed. No further action is required.

Azure Default Path Access Test:

Test Objective:
To verify whether default Azure App Service management paths (e.g., /debugconsole, /.env, /scm) are publicly accessible. These paths, if exposed, could reveal sensitive system or platform information.

Test Implementation:
We manually tested a list of known Azure default paths by visiting them via a browser. Each path was accessed directly through the live website and its HTTP response was observed.

Testing Result:
All tested paths returned either 404 Not Found or 403 Forbidden, indicating they are not publicly accessible. No Azure debug consoles, configuration files, or platform-specific resources were exposed.

DOM-based XSS Test:

Test Objective:
To verify whether the website's JavaScript code processes data from the URL (such as query parameters) in a way that could lead to DOM-based Cross-Site Scripting (XSS). Even in a static site with no forms or uploads, improper handling of window.location or document.location.search can lead to front-end script injection.
Test Implementation:
We manually tested several common XSS payloads by appending them to the URL as query parameters. Each payload was evaluated by observing whether any alerts were triggered, or whether payload content was reflected in the DOM without proper encoding.

Payloads used intesting::
?q=<script>alert(1)</script>
?q="><script>alert(1)</script>
?q=';alert(1)//
?q=<img src=x onerror=alert(1)>
?q=<svg/onload=alert(1)>
?q=<body onload=alert(1)>
?q=<iframe src=javascript:alert(1)>
Testing Result:
None of the payloads executed in the browser. No alert dialogs were triggered, and no payload content was visibly reflected in the DOM. Additionally, a review of the JavaScript source code confirmed that location.search or window.location are not used to write directly to the DOM. Therefore, we conclude that the application is not vulnerable to DOM-based XSS.

Database Server Security Test:

Test Objective:
To assess the security of the Azure-hosted MySQL database service, including connection

encryption, access control, and audit configurations.

Test Implementation:
During the initial deployment of the database, Azure enforced strict security requirements which we successfully fulfilled:

- **Mandatory SSL encryption** with certificate validation
- **Firewall-level IP access control**, allowing only authorized client connections
- **User authentication enforcement**, preventing anonymous access
- **System logs and query logs are automatically enabled** by default
- **No external access is possible** without explicit permission

Testing Result:
All key security controls provided by Azure Database for MySQL are active and functioning as expected. Since the environment is protected at the platform level and has already been verified during deployment, no further manual penetration testing was deemed necessary.

Reference:

1. Mozilla. (n.d.). *Observatory by Mozilla*. Retrieved April 10, 2025, from https://observatory.mozilla.org

2. Snyk. (n.d.). *Snyk Advisor: JavaScript package security insights*. Retrieved April 10, 2025, from https://snyk.io/advisor

3. Google. (n.d.). *Chrome DevTools*. Retrieved April 10, 2025, from https://developer.chrome.com/docs/devtools/

4. OWASP Foundation. (2021). *OWASP ZAP: The Zed Attack Proxy*. Retrieved from https://www.zaproxy.org/

5. PortSwigger Ltd. (n.d.). *Burp Suite – Web vulnerability scanner*. Retrieved from https://portswigger.net/burp

6. Nmap.org. (n.d.). *Nmap: Network Mapper*. Retrieved from https://nmap.org/
7. OWASP Foundation. (n.d.). *DOM based XSS Prevention Cheat Sheet*. Retrieved from https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html

8. Microsoft. (n.d.). *Azure Database for MySQL - Security Overview*. Retrieved from https://learn.microsoft.com/en-us/azure/mysql/